# 设置您的计算机（Windows + WSL2）

**请在第一次实验前**完成此步骤。预留 45 分钟，大部分时间用于等待下载。如果出现任何错误，请记下完整的错误信息并带到实验室——不要浪费两个小时去猜测。

macOS 和 Linux 用户：请跳至文末。

## 为什么选择 WSL2 而不是普通的 Windows？

本课程讲解指针，而查找指针错误的最佳工具是 AddressSanitizer。它能*准确地*告诉你哪一行代码写入了数组末尾之外，而不是让你的程序在十分钟后莫名其妙地崩溃。

在 Windows 上的 MinGW/MSYS2 GCC 中，AddressSanitizer 不可用。WSL2 提供了一个真正的 Linux 环境，只需一条命令即可使用 AddressSanitizer `g++` 、 MSYS2 GCC `gdb` 和 `valgrind` AddressSanitizer，并且与实验室机器完全一致。您仍然可以在 Windows 上的 VS Code 中编辑文件；只有编译器在 Linux 端运行。

## 步骤 1 — 安装 WSL2

以管理员身份打开**PowerShell**并运行：

```
wsl --install -d Debian
```

出现提示时请重启。首次启动 Debian 时，系统会要求您选择用户名和密码。**此密码并非您的 Windows 密码**，您之后还需要用到它 `sudo` ，因此请务必妥善保管。

检查版本：

```
wsl -l -v
```

你想要 `VERSION 2` ……如果显示 1，运行 `wsl --set-version Debian 2` ……

## 步骤 2 — 在 Debian 系统中安装工具链

打开**Debian**应用程序（不是 PowerShell），然后运行：

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential gdb cmake git valgrind clang-format
```

检查一下你得到了什么：

```
g++ --version      # want 11 or newer
cmake --version    # want 3.20 or newer
```

## 步骤 3 — 在 Windows 上安装 VS Code

从https://code.visualstudio.com/下载。安装过程中勾选“添加到 PATH”。

然后安装**WSL扩展**：打开 VS Code，按 `Ctrl+Shift+X` ，搜索 `WSL` ，安装 `ms-vscode-remote.remote-wsl` 。

## 第四步——告诉 Git 你是谁

**以前没用过 Git？** `docs/GIT_BASICS.md` **现在就**阅读——下面的命令会更容易理解，而且无论如何，在你第一次提交之前，你都需要阅读本教程的其余部分。

在 Debian 终端中：

```
git config --global user.name  "Your Name"
git config --global user.email "your.email@university.edu"
git config --global init.defaultBranch main
```

## 第五步——获取启动项目

本课程的代码托管在**自托管的 Gitea 实例**上，而不是 GitHub 上。入门项目模板位于 https://git.pdulab.net/pdu/2440099。

**重要提示：克隆到 Linux 文件系统，而不是其他文件系统** `/mnt/c/...` 。在其他文件 系统上构建速度 `/mnt/c` 会慢 5-10 倍，因为每次文件访问都会在 Windows 和 Linux 之间进行。

```
cd ~
mkdir -p 2440099 && cd 2440099
git clone https://git.pdulab.net/pdu/2440099 starter
cd starter
code .
```

最后一条命令会在 Windows 系统上打开 VS Code，并连接到 WSL。窗口左下角应该显示“**WSL: Debian**”。如果没有显示，说明你编辑的文件副本有误。

VS Code 会提示您安装推荐的扩展程序。请选择“是”。

## 步骤 6 — 构建并运行

在 VS Code 终端（ `Ctrl+`` 它已经在 WSL 内部）：

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug
./build/debug/bin/game
```

@ 你应该看到所有测试都通过了，然后会进入一个可以自由走动的房间。按下按钮 `Q` 退出。

您也可以使用状态栏中的 CMake 工具按钮，而不是键入这些命令。

## 步骤 7 — 安装 Cline，您的 AI 助手（暂时不要使用它）

本课程的AI辅助功能通过VS Code扩展程序**Cline**运行，使用 **您自己的DeepSeek API密钥**——即"自带密钥"（BYOK）。没有共享的部门账户，也没有中央消费上限；您需要自行注册DeepSeek账户，充值少量资金，Cline会从该余额中支出。此步骤预计需要15分钟，大部分时间将用于等待短信验证码和付款确认。

1. **安装 Cline。**在 VS Code 中 `Ctrl+Shift+X` ，搜索**Cline**并安装它（ `saoudrizwan.claude-dev` ）。它不在本仓库步骤 5 推荐的扩展列表中——该列表特意保持精简，这样 VS Code 首先要求你安装的就不会是一个你三周内都不能使用的 AI 工具——所以你需要自己搜索它，就在步骤 7 中。

2. **注册一个DeepSeek账号。**在浏览器中访问 https://platform.deepseek.com （无需VPN——这是一项中国境内的服务）。点击"**注册**"。

   - **如果您有中国手机号码，请使用它。**输入号码，请求短信验证码，输入验证码。这是最快捷的方式，也是本课程大多数学生会使用的方式。
   - 如果您没有中国手机号码，请改用电子邮件地址注册——点击电话号码栏附近的"使用电子邮件"/"邮箱注册"链接，验证发送到您收件箱的确认码。
   - 设置一个真实有效的密码并将其保存在密码管理器中。此账户将持有少量现金余额，因此请像对待其他任何有资金的账户一样对待它。

3. **为您的账户充值。**DeepSeek的 API 采用预付费模式：您预先充值，每次请求都会从中扣除——除了您选择充值的金额外，不会自动收取任何费用，也不会产生意外账单。在平台左侧边栏中，找到"**充值**"（可能显示为 "**充值**"或位于右上角的账户余额），然后充值。

   - **根据下方步骤 5 中的模型，本课程的预期使用量下，20-30 元人民币足以支付整个学期的费用**——本课程自身的成本模型显示，在此模型下，每位学生一个学期的费用约为 1-2 美元（远低于 20 元人民币）。您无需加载更多，如果您进行大量额外的实验，可以稍后添加更多。
   - 如果您身处中国，可以使用支付**宝或微信支付**——充值页面直接支持这两种支付方式。如果您无法使用支付宝或微信支付，也可以使用银行卡支付。
   - 如果充值页面显示的布局与此处描述的布局不同（这些平台会定期重新设计其计费页面），请查找标题中提及"余额"、"余额"、"充值"或"充值"的任何页面。

4. **创建 API 密钥。**仍在 DeepSeek 平台中， 在左侧边栏（有时位于"开发者"或"API"部分下方）找到**API 密钥。点击**"**创建新 API 密钥**"，为其命名以便日后识别（例如 `cline-2440099` ），然后确认。DeepSeek**会显示完整的密钥一次**——在关闭对话框之前，请立即复制它（通常会有一个复制图标按钮）。如果您在未复制的情况下关闭了对话框，也不会丢失任何重要信息；只需删除该密钥并创建一个新密钥即可。

   - 请像对待密码一样对待此密钥：任何拥有它的人都可以随意使用您的余额。下一步请直接将其粘贴到 Cline 中，切勿粘贴到其他任何地方。**切勿将您的 API 密钥放在此代码库中的任何文件**、聊天消息或任何公共网站上——如果您发现即将提交的文件中包含密钥，请立即停止操作并在提交前将其删除。

5. **使用您的钥匙配置 Cline，并选择型号。**打开 Cline 的设置（侧面板中的齿轮图标）。

   - **API 提供商：**从提供商下拉菜单中选择**DeepSeek** 。
   - **API密钥：**粘贴您刚刚复制的密钥。
   - **型号：**选择**DeepSeek V4.1 Flash**（可能以型号 ID 列出 `deepseek-flash` ）。这是本课程预算所依据的低廉价格、快速版本——更大型/更高级的 DeepSeek 型号每个令牌的价格更高，但对于你本学期要提出的问题类型来说并没有任何优势，因此没有理由选择更昂贵的型号。
   - 如果**DeepSeek**没有作为预设提供程序出现（Cline 的提供程序列表会随着时间而变化），请改为选择**OpenAI Compatible**，将基本 URL 设置为 `https://api.deepseek.com` ，粘贴您的密钥，然后手动输入模型名称。
   - 保存。Cline 将密钥存储在 VS Code 自身的秘密存储空间中，而不是项目文件中——这就是为什么将其粘贴到这里（而不是其他任何地方）是安全的。

6. **请确认 Cline 是否正确读取了本课程的规则——在第四周开始依赖它之前，请先执行一次此操作。Cline 会** `.clinerules` 自动读取工作区根目录下的一个名为 `<your_classes_name>` 的文件；它**不会** `AGENTS.md` 直接读取 `<your_classes_name>` 文件（某些*其他*工具会自行读取 `<your_classes_name>` `AGENTS.md` 文件——本仓库通过`<your_classes_name>` 保持两者同步，但 Cline 只读取 `<your_classes_name> ` 文件）。要检查它是否正常工作，请打开 Cline 面板并发送一条消息——即使在第 1-3 周，发送这条消息也是可以的，因为您是在测试该工具，而不是将其用于课程作业： `scripts/sync_ai_rules.sh .clinerules`

   > 在这个代码仓库中，你必须遵循哪些 C++ 语言规则？

   一个正常工作的配置会给出诸如"无 STL 容器"、"无智能指针"、"仅限 C++14"之类的答案——也就是实际的配置内容 `AI_RULES.md` 。如果它给出的是通用答案：

   - 确认文件 `.clinerules` 位于您打开的文件夹的最顶部（ `ls -a ~/2440099/starter/.clinerules` ）——而不是在文件夹内部 `docs/` 或 `src/` 。
   - 确认 VS Code 的工作区根目录是该文件夹本身，而不是它的父文件夹—— `code .` 从 `starter/` 步骤 5 的内部操作会自动正确执行此操作。
   - 重新加载窗口（ `Ctrl+Shift+P` → `Developer: Reload Window` ），以便 Cline 重新扫描工作区。
   - 检查 Cline 的设置，看看是否有"自定义说明"开关被关闭。
   - 如果 Cline 报告 API 错误而不是答案，请重新检查步骤 5——输入错误的密钥或 DeepSeek 余额为空都会导致此步骤失败，而不会静默失败。

7. 步骤 6 完成后，暂时不要向 Cline 发送任何其他内容。根据本课程的分阶段人工智能策略（ `AI_RULES.md` ），第 1-3 周人工智能功能将关闭，并通过闭卷实验进行检查。现在安装和验证人工智能功能，只是为了确保它在第 4 周准备就绪。

## 如果出了什么问题

**`wsl --install` 提示该功能未启用。**可能是您的 BIOS 中虚拟化功能已关闭，或者 Hyper-V 已禁用。请搜索您的笔记本电脑型号加上"启用 BIOS 虚拟化"。

**`code .` 提示找不到命令。**VS Code 安装时未启用"添加到 PATH"选项。请重新安装，或手动启动 VS Code 并使用 `Ctrl+Shift+P` → `WSL: Connect to WSL` 。

**构建成功，但调试器始终无法在断点处停止。**您可能正在运行 Release 预设。请将 CMake 工具状态栏切换回 `debug` .

**方向键输入的是文本 `^[[A` 而不是移动。**您当前是通过管道或 VS Code 的输出面板运行程序，而不是通过真正的终端。请从集成终端运行程序。

**所有操作都非常慢。**您已将项目克隆到 [此处应填写克隆位置] `/mnt/c/Users/...` 。请将项目移动到 [此处应填写 `~/2440099/` 目标位置] 并重新构建。

## macOS

```
xcode-select --install
brew install cmake git clang-format
```

然后按照上述步骤 4-7 操作。使用**CodeLLDB**扩展而不是 GDB 启动配置。

## Linux

```
sudo apt install -y build-essential gdb cmake git valgrind clang-format
```

然后按照上述步骤 4-7 进行操作。